

LLM4Unroll 项目完整实施方案

Language-Guided Verifiable Algorithm Unrolling for Operator-Splitting
Optimisers

面向组合优化专业求解器的 DSL、算法注册表、优化评价器与
Benchmark 方案

科研计划技术蓝图

2026 年 4 月 30 日

摘要

本文档给出一套可直接转化为代码构建任务的完整方案。目标是构建一个 **LLM4Unroll** 系统：使用大语言模型作为离线算法设计器，在受控 DSL 中自动生成、改写和进化 operator-splitting 优化算法的展开策略，包括 PDHG、ADMM、FISTA/PGD、Douglas–Rachford、ALM/SQP 子问题、PCG 内核以及 LNS-relaxation-repair 求解器模块。系统不让 LLM 直接求解组合优化问题，也不让 LLM 在线逐步决策；而是让 LLM 生成可执行、可验证、可评估的 update policy、stepsize schedule、restart rule、penalty update、residual balancing、relaxation repair 等组件。生成候选经过静态 verifier、运行时 safety guard、optimisation evaluator 和跨规模 benchmark 筛选，最终形成可以插入专业组合优化求解器的 algorithm-unrolling component。

文档包含：(1) DSL 设计；(2) 原算法与可改组件的统一注册表文件设计；(3) optimisation evaluator 设计；(4) benchmark 获取与使用；(5) baseline 体系；(6) 项目目录与代码实现计划；(7) 理论目标；(8) 面向 coding agent 的超级提示词草案。

目录

1 项目定位与核心判断

1.1 研究目标

本项目的核心目标是构建一个面向组合优化专业求解器的 **语言引导可验证算法展开框架**。形式上，给定一族组合优化问题

$$\min_{x \in \mathcal{X}} c^\top x \quad \text{s.t.} \quad Ax \leq b, \quad x_i \in \mathbb{Z} \text{ or } \{0, 1\}, \quad (1)$$

专业求解器通常会调用 LP/QP 松弛、拉格朗日松弛、ADMM/PDHG 一阶模块、cut separation、heuristic repair、large neighbourhood search 等内部组件。本文档关注其中可被 algorithm unrolling 化的部分：

$$z_{k+1} = \mathcal{T}_{a_k, \theta_k}(z_k; P), \quad k = 0, \dots, K-1, \quad (2)$$

其中 P 是问题实例， z_k 是算法状态， a_k 是更新算子类型， θ_k 是步长、惩罚参数、动量、restart、scaling、neighbourhood 强度等可调组件。

本文档建议的研究命题是：

LLM 不直接求解组合优化问题，而是在可验证 DSL 中离线生成 unrolled optimiser 的策略程序；通过 optimisation evaluator 和专业求解器 benchmark 自动筛选，使其成为可插入 MILP/CP-SAT/PDLIP/OSQP/SCIP 流程的加速组件。

1.2 为什么不是直接做 LLM 求解器

直接让 LLM 输出组合优化解或逐步控制 solver 有三个问题：

1. **不可验证**：LLM 生成的自然语言推理很难保证可行性、收敛性或稳定性。
2. **不可复现**：prompt、温度、API 版本和采样噪声可能导致结果波动。
3. **难以超越强 baseline**：专业求解器如 SCIP、HiGHS、OR-Tools、Gurobi/CPLEX 已经非常强，单纯的 LLM agent 很难正面竞争。

因此，本项目采用 **LLM-as-offline-algorithm-designer**：LLM 只生成策略程序，最终结果由 evaluator、verifier 和 solver benchmark 决定。该范式与 FunSearch、EoH、ReEvo、LLM-LNS 等工作在思想上相近，但本文档的重点不是 heuristic generation，而是 **verifiable algorithm unrolling**。

1.3 项目最小闭环

第一阶段建议形成如下最小闭环：

$$\text{EoH/ReEvo 搜索框架} + \text{PDHG/ADMM/FISTA DSL} + \text{HiGHS/OSQP/SCIP baseline} + \text{MIPLIB/LLM} \quad (3)$$

最小可发表版本不追求“击败所有商业求解器”，而是证明：在给定训练预算和同等 evaluator 条件下，LLM 生成的可验证 unrolled policy 能够稳定优于手工 schedule、random/evolution/BO 搜索以及常规 learned schedule，并能在更大实例上保持较低失败率。

2 系统总架构

2.1 总体流程

建议系统命名为 `llm4unroll`。总体流程如下：

1. **Algorithm Registry**: 把所有可展开原算法、状态变量、可改组件、默认安全范围归纳到一个统一文件。
2. **DSL Layer**: 定义 LLM 可生成的策略程序语法，包括 features、actions、guards、update rules。
3. **LLM Search Layer**: 复用 EoH/ReEvo/FunSearch 式进化搜索，生成候选 policy。
4. **Verifier Layer**: 静态检查候选程序是否符合 DSL、安全边界和 solver 接口。
5. **Optimisation Evaluator**: 运行候选算法，返回 objective gap、residual、violation、runtime、fail rate、query cost 等指标。
6. **Benchmark Layer**: 在 synthetic、LP/QP relaxation、MIPLIB 子集、LLM-LNS MILP 数据、Ecole/SCIP 实例上测试。
7. **Baseline Layer**: 比较 vanilla algorithm、hand-crafted adaptive rule、random/BO/evolution、learned-controller、LLM heuristic baselines。
8. **Report Layer**: 自动生成 result table、curve、ablation 和 failure analysis。

2.2 推荐项目目录

```
llm4unroll/
  README.md
  pyproject.toml
  configs/
```

```
default.yaml
pdhg_miplib_rootlp.yaml
admm_qp_relaxation.yaml
fista_lasso.yaml
src/llm4unroll/
registry/
  algorithm_registry.py
  problem_registry.py
  baseline_registry.py
dsl/
  schema.py
  primitives.py
  parser.py
  verifier.py
  guards.py
  transpiler.py
algorithms/
  base.py
  pdhg.py
  admm.py
  fista.py
  douglas_rachford.py
  alm.py
  pcg.py
  lns_repair.py
search/
  eoh_adapter.py
  reevo_adapter.py
  funsearch_adapter.py
  llm_client.py
  population.py
  prompts.py
evaluator/
  optimisation_evaluator.py
  metrics.py
  safety.py
  budget.py
  report.py
solvers/
  highs_interface.py
```

```
osqp_interface.py
pyscipopt_interface.py
ortools_interface.py
ecole_interface.py
benchmarks/
  miplib.py
  llm_lns_data.py
  ecole_instances.py
  synthetic_lp_qp.py
  graph_relaxations.py
experiments/
  run_search.py
  run_baselines.py
  run_ablation.py
  run_ood.py
external/
  LLM-LNS/
  EoH/
  reevo/
  PDHG-Net/
  osqp/
  HiGHS/
  PySCIPOpt/
  ecole/
  or-tools/
  heurigym/
data/
  miplib/
  llm_lns/
  synthetic/
results/
  logs/
  candidates/
  tables/
  figures/
scripts/
  setup_external.sh
  download_miplib.sh
  prepare_llm_lns_data.sh
  reproduce_phase1.sh
```

3 DSL 设计

3.1 DSL 的设计原则

DSL 是本项目的核心。它必须满足五个要求：

1. **表达力足够**：能表达 operator-splitting 算法里的步长、惩罚、restart、residual balancing、damping、normalisation、projection 等策略。
2. **安全边界明确**：所有参数必须有上下界，所有除法必须有 ϵ ，所有矩阵操作必须限制维度和稀疏性。
3. **可静态检查**：LLM 生成的程序必须能被 AST/parser 检查，不允许任意文件读写、网络访问、动态 import、exec/eval。
4. **可运行时保护**：即便静态检查通过，运行时也要有 NaN/Inf 检查、residual explosion 检查、fallback rule。
5. **可理论分析**：DSL 生成的策略应落入某些 safe policy class，便于证明 boundedness、non-expansiveness、residual stability 或 convergence-preserving 性质。

3.2 DSL 中的状态变量

统一状态对象记为 s_k ，包含下列字段：

字段	含义
k	当前展开层数/迭代数
obj、obj_prev	当前和上一轮目标函数值
gap	primal-dual gap 或相对 objective gap
r_p、r_d	primal residual 和 dual residual
r_p_norm、r_d_norm	residual 范数
violation	约束违反程度，例如 $\ [Ax - b]_+ \ $
tau、sigma	PDHG 或 proximal 算法的 primal/dual stepsize
rho	ADMM/ALM/OSQP 类算法的 penalty parameter
momentum	FISTA/Nesterov 动量参数
scale_x、scale_y	diagonal scaling 或 normalization 系数
stagnation_count	连续多少轮改善不足
time_elapsed	当前候选的运行时间

`instance_features` 问题规模、稀疏度、变量/约束比例、整数变量比例等实例特征

3.3 DSL 中的动作空间

LLM 输出的 policy 不是直接更新变量 x, y ，而是返回 actions。动作空间包括：

动作	说明
<code>set_tau(value)</code>	设置 primal stepsize, 必须满足 $\tau_{\min} \leq \tau \leq \tau_{\max}$
<code>set_sigma(value)</code>	设置 dual stepsize, 必须满足 $\sigma_{\min} \leq \sigma \leq \sigma_{\max}$
<code>scale_tau(factor)</code>	乘法调整 τ
<code>scale_sigma(factor)</code>	乘法调整 σ
<code>set_rho(value)</code>	设置 ADMM/ALM penalty
<code>scale_rho(factor)</code>	residual balancing 中的 penalty 调整
<code>restart()</code>	重置 momentum、extrapolation 或 conjugacy memory
<code>damp(alpha)</code>	对更新做 damping, $z_{k+1} \leftarrow (1 - \alpha)z_k + \alpha\tilde{z}_{k+1}$
<code>clip_update(max_norm)</code>	限制更新范数
<code>switch_operator(name)</code>	在允许的混合模板中切换 PGD/PDHG/ADMM/repair operator
<code>fallback()</code>	回到默认安全规则

3.4 DSL 的 Python 数据结构

下面是建议实现的 `schema.py`。

```
# src/llm4unroll/dsl/schema.py
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Dict, List, Literal, Optional, Tuple, Any

AlgorithmName = Literal[
    "PGD", "FISTA", "PDHG", "ADMM", "DRS", "ALM", "PCG", "LNS_REPAIR"
]

ActionType = Literal[
    "set_tau", "set_sigma", "scale_tau", "scale_sigma",
    "set_rho", "scale_rho", "set_momentum",
    "restart", "damp", "clip_update", "switch_operator", "fallback"
]
```

```
@dataclass(frozen=True)
class StateSpec:
    allowed_features: List[str]
    required_features: List[str]
    feature_bounds: Dict[str, Tuple[float, float]] = field(default_factory=dict)

@dataclass(frozen=True)
class ActionSpec:
    name: ActionType
    lower: Optional[float] = None
    upper: Optional[float] = None
    allowed_values: Optional[List[str]] = None

@dataclass(frozen=True)
class SafetySpec:
    max_policy_lines: int = 80
    max_runtime_ms: int = 20
    allow_loops: bool = False
    allow_imports: bool = False
    require_fallback: bool = True
    nan_policy: str = "fallback"
    residual_explosion_factor: float = 1e3
    min_tau: float = 1e-8
    max_tau: float = 1e8
    min_sigma: float = 1e-8
    max_sigma: float = 1e8
    min_rho: float = 1e-8
    max_rho: float = 1e8

@dataclass(frozen=True)
class PolicySpec:
    algorithm: AlgorithmName
    state: StateSpec
    actions: List[ActionSpec]
    safety: SafetySpec
    description: str

@dataclass
class PolicyOutput:
    actions: List[Tuple[ActionType, Any]]
```

```
reason: str = ""
diagnostics: Dict[str, Any] = field(default_factory=dict)
```

3.5 LLM 生成策略的函数签名

为避免 LLM 生成任意 solver，所有候选策略都必须符合统一函数签名：

```
def policy(state: dict) -> dict:
    """
    Input:
        state: normalised algorithm state. It only contains whitelisted fields.
    Output:
        A dictionary with keys:
            actions: list of tuples, e.g. [("scale_rho", 2.0), ("restart", True)]
            reason: short text for logging only
    Constraints:
        - no import, no file I/O, no network
        - no while/for loops in phase 1
        - no mutation of global variables
        - must return fallback-safe actions when uncertainty is high
    """
    ...
```

3.6 DSL 示例：ADMM penalty policy

```
def policy(state: dict) -> dict:
    rp = state["r_p_norm"]
    rd = state["r_d_norm"]
    rho = state["rho"]
    eps = 1e-12

    ratio = rp / (rd + eps)
    actions = []

    if ratio > 8.0:
        actions.append(("scale_rho", 1.7))
    elif ratio < 0.125:
        actions.append(("scale_rho", 0.6))
    else:
        actions.append(("scale_rho", 1.0))
```

```

if state.get("stagnation_count", 0) >= 8:
    actions.append(("damp", 0.75))

return {
    "actions": actions,
    "reason": "Residual-balanced rho update with stagnation damping."
}

```

3.7 DSL 示例: PDHG stepsize 与 restart policy

```

def policy(state: dict) -> dict:
    rp = state["r_p_norm"]
    rd = state["r_d_norm"]
    gap = state.get("gap", 1.0)
    tau = state["tau"]
    sigma = state["sigma"]
    obj = state.get("obj", 0.0)
    obj_prev = state.get("obj_prev", obj)
    eps = 1e-12

    balance = rp / (rd + eps)
    actions = []

    if balance > 5.0:
        actions += [("scale_tau", 0.85), ("scale_sigma", 1.10)]
    elif balance < 0.2:
        actions += [("scale_tau", 1.10), ("scale_sigma", 0.85)]
    else:
        actions += [("scale_tau", 1.02), ("scale_sigma", 1.02)]

    # Conservative restart if objective/gap gets worse and residual is not
    # balanced.
    if obj > obj_prev and (balance > 10.0 or balance < 0.1):
        actions.append(("restart", True))
        actions.append(("damp", 0.7))

    if gap > 1e2:
        actions.append(("clip_update", 10.0))

```

```
return {"actions": actions, "reason": "Adaptive PDHG balancing rule."}
```

4 原算法注册表：所有可更改适用于 Unroll 的算法归纳到一个文件

4.1 设计目标

用户明确要求“把所有可更改适用于 unroll 的原算法归纳到一个文件里面, LLM4Unroll 方案可以直接从这里开始设计”。本项目建议把该文件命名为：

src/llm4unroll/registry/algorithm_registry.py

该文件不是算法实现本体，而是 **算法元信息注册表**：告诉 LLM、verifier、evaluator 哪些算法能展开、状态是什么、可改组件是什么、安全边界是什么、默认 baseline 是什么。

4.2 算法族选择

第一版建议包括以下算法：

算法	主用途	可让 LLM 设计的组件
PGD	简单约束/投影松弛	stepsize、projection schedule、damping、clip
FISTA	稀疏/光滑 + 非光滑复合问题	momentum、restart、stepsize、monotone correction
PDHG	LP、saddle-point、约束松弛	τ, σ, θ 、restart、residual balancing、scaling
ADMM	QP、consensus、分解松弛	ρ update、over-relaxation、residual balancing、damping
DRS	feasibility/QP/proximal relaxation	parameter、operator ordering、restart
ALM	约束优化/拉格朗日松弛	penalty schedule、multiplier damping、feasibility repair
PCG	QP/KKT/二阶子问题内核	restart、truncation、preconditioner selection、stopping rule

LNS-REPAIR	MILP	heuris-	neighbourhood size、fixing policy、re-
	tic/repair		laxation repair schedule

4.3 algorithm_registry.py 示例

```
# src/llm4unroll/registry/algorithm_registry.py
from llm4unroll.dsl.schema import PolicySpec, StateSpec, ActionSpec, SafetySpec

ALGORITHM_REGISTRY = {
    "FISTA": PolicySpec(
        algorithm="FISTA",
        description=(
            "Unrolled proximal-gradient/FISTA template for composite convex "
            "objectives. LLM may design stepsize, restart, momentum damping."
        ),
        state=StateSpec(
            allowed_features=[
                "k", "obj", "obj_prev", "grad_norm", "step_norm",
                "momentum", "stagnation_count", "instance_features"
            ],
            required_features=["k", "obj", "obj_prev", "grad_norm", "momentum"],
        ),
        actions=[
            ActionSpec("set_tau", 1e-8, 1e4),
            ActionSpec("scale_tau", 0.1, 10.0),
            ActionSpec("set_momentum", 0.0, 0.999),
            ActionSpec("restart"),
            ActionSpec("damp", 0.05, 1.0),
            ActionSpec("clip_update", 1e-8, 1e4),
            ActionSpec("fallback"),
        ],
        safety=SafetySpec(require_fallback=True),
    ),

    "PDHG": PolicySpec(
        algorithm="PDHG",
        description=(
            "Primal-dual hybrid gradient / Chambolle-Pock style template "
            "for LP, saddle-point and convex constrained relaxations."
        )
    )
}
```

```

    ),
    state=StateSpec(
        allowed_features=[
            "k", "obj", "obj_prev", "gap", "r_p_norm", "r_d_norm",
            "violation", "tau", "sigma", "theta", "stagnation_count",
            "instance_features"
        ],
        required_features=["k", "r_p_norm", "r_d_norm", "tau", "sigma"],
    ),
    actions=[
        ActionSpec("set_tau", 1e-8, 1e8),
        ActionSpec("set_sigma", 1e-8, 1e8),
        ActionSpec("scale_tau", 0.1, 10.0),
        ActionSpec("scale_sigma", 0.1, 10.0),
        ActionSpec("restart"),
        ActionSpec("damp", 0.05, 1.0),
        ActionSpec("clip_update", 1e-8, 1e6),
        ActionSpec("fallback"),
    ],
    safety=SafetySpec(
        require_fallback=True,
        residual_explosion_factor=1e3,
        min_tau=1e-8, max_tau=1e8,
        min_sigma=1e-8, max_sigma=1e8,
    ),
),

"ADMM": PolicySpec(
    algorithm="ADMM",
    description=(
        "ADMM / OSQP-style operator-splitting template for convex QP, "
        "consensus relaxations and decomposition subproblems."
    ),
    state=StateSpec(
        allowed_features=[
            "k", "obj", "obj_prev", "gap", "r_p_norm", "r_d_norm",
            "violation", "rho", "alpha", "stagnation_count",
            "instance_features"
        ],
        required_features=["k", "r_p_norm", "r_d_norm", "rho"],

```

```

    ),
    actions=[
        ActionSpec("set_rho", 1e-8, 1e8),
        ActionSpec("scale_rho", 0.1, 10.0),
        ActionSpec("damp", 0.05, 1.0),
        ActionSpec("restart"),
        ActionSpec("clip_update", 1e-8, 1e6),
        ActionSpec("fallback"),
    ],
    safety=SafetySpec(require_fallback=True, min_rho=1e-8, max_rho=1e8),
),

"DRS": PolicySpec(
    algorithm="DRS",
    description=(
        "Douglas-Rachford splitting template for feasibility, proximal "
        "composition and selected QP/convex relaxation modules."
    ),
    state=StateSpec(
        allowed_features=[
            "k", "obj", "obj_prev", "gap", "residual_norm",
            "lambda_relax", "stagnation_count", "instance_features"
        ],
        required_features=["k", "residual_norm", "lambda_relax"],
    ),
    actions=[
        ActionSpec("damp", 0.05, 1.0),
        ActionSpec("restart"),
        ActionSpec("clip_update", 1e-8, 1e6),
        ActionSpec("fallback"),
    ],
    safety=SafetySpec(require_fallback=True),
),

"ALM": PolicySpec(
    algorithm="ALM",
    description=(
        "Augmented Lagrangian template for constrained optimisation and "
        "Lagrangian relaxations used inside CO solvers."
    ),
    ),

```

```

state=StateSpec(
    allowed_features=[
        "k", "obj", "obj_prev", "gap", "violation", "rho",
        "multiplier_norm", "stagnation_count", "instance_features"
    ],
    required_features=["k", "violation", "rho"],
),
actions=[
    ActionSpec("set_rho", 1e-8, 1e8),
    ActionSpec("scale_rho", 0.1, 10.0),
    ActionSpec("damp", 0.05, 1.0),
    ActionSpec("restart"),
    ActionSpec("fallback"),
],
safety=SafetySpec(require_fallback=True, min_rho=1e-8, max_rho=1e8),
),

"PCG": PolicySpec(
    algorithm="PCG",
    description=(
        "Preconditioned conjugate-gradient kernel for SPD systems, "
        "KKT reduced systems and QP/Newton subproblems. Not the main "
        "unrolling object in phase 1, but kept as an inner-solver module."
    ),
    state=StateSpec(
        allowed_features=[
            "k", "residual_norm", "residual_ratio", "direction_curvature",
            "precond_quality", "stagnation_count", "instance_features"
        ],
        required_features=["k", "residual_norm"],
    ),
    actions=[
        ActionSpec("restart"),
        ActionSpec("damp", 0.05, 1.0),
        ActionSpec("clip_update", 1e-8, 1e6),
        ActionSpec("fallback"),
    ],
    safety=SafetySpec(require_fallback=True),
),

```

```

"LNS_REPAIR": PolicySpec(
    algorithm="LNS_REPAIR",
    description=(
        "Large-neighbourhood-search plus continuous relaxation repair "
        "template for MILP. LLM may design fixing ratio, neighbourhood "
        "schedule and relaxation-based repair triggers."
    ),
    state=StateSpec(
        allowed_features=[
            "k", "incumbent_obj", "best_bound", "mip_gap", "lp_gap",
            "fractionality", "violation", "neighbourhood_size",
            "stagnation_count", "instance_features"
        ],
        required_features=["k", "mip_gap", "fractionality", "
            neighbourhood_size"],
    ),
    actions=[
        ActionSpec("damp", 0.05, 1.0),
        ActionSpec("restart"),
        ActionSpec("fallback"),
        # In implementation, add specific actions such as:
        # set_fixing_ratio, set_neighbourhood_size, trigger_repair.
    ],
    safety=SafetySpec(require_fallback=True),
),
}

```

4.4 为什么把 CG 放在注册表但不作为第一主线

CG/PCG 对 SPD 线性系统非常重要，但它不像 ADMM/PDHG/FISTA 一样天然具有交替 primal-dual 或 proximal layer 结构。因此第一阶段不建议把 CG 作为主要 unrolling 算法，而应将其作为 QP/KKT/二阶子问题的内核，用于 restart、truncation、preconditioning policy 的附加实验。这样既保留 CG 特色，又避免论文主线被“CG 难展开”拖住。

5 把 evaluator 换成 Optimisation Evaluator

5.1 从 heuristic evaluator 到 optimisation evaluator

EoH/ReEvo/FunSearch 里的 evaluator 通常是：运行一个候选 heuristic，返回目标值或解质量。本项目需要把 evaluator 改成 **optimisation evaluator**，即对候选 unrolled policy 执行以下流程：

1. 加载 benchmark instance；
2. 根据算法注册表初始化 solver state；
3. 在固定 iteration/time/query budget 下运行候选 policy；
4. 每层调用 base operator，例如 PDHG update、ADMM update、FISTA update；
5. 记录 objective gap、primal/dual residual、constraint violation、integrality gap、runtime、memory、failure flag；
6. 对多实例、多随机种子、多规模取稳健统计量；
7. 输出一个 scalar score 和详细 diagnostics。

5.2 统一评分函数

候选 policy 的评分建议采用多目标加权加惩罚：

$$\begin{aligned}
 \text{Score}(\pi) = & -w_1 \cdot \text{median}_{P \in \mathcal{D}} \log(1 + \text{gap}_P) \\
 & -w_2 \cdot \text{median}_{P \in \mathcal{D}} \log(1 + \text{vio}_P) \\
 & -w_3 \cdot \text{median}_{P \in \mathcal{D}} \log(1 + \text{time}_P) \\
 & -w_4 \cdot \text{fail_rate}(\pi) \\
 & -w_5 \cdot \text{complexity}(\pi).
 \end{aligned} \tag{4}$$

其中 $\text{complexity}(\pi)$ 可以是 policy 行数、分支数、动作数、LLM 查询成本或 evaluator 运行成本。

5.3 Evaluator 接口设计

```
# src/llm4unroll/evaluator/optimisation_evaluator.py
from dataclasses import dataclass, field
from typing import Dict, List, Any, Optional
import time
import math
```

```

@dataclass
class EvalMetrics:
    objective: float
    best_bound: Optional[float]
    gap: float
    primal_residual: float
    dual_residual: float
    violation: float
    integrality_gap: Optional[float]
    runtime: float
    iterations: int
    memory_mb: Optional[float]
    failed: bool
    failure_reason: str = ""
    diagnostics: Dict[str, Any] = field(default_factory=dict)

@dataclass
class EvalResult:
    score: float
    metrics_by_instance: Dict[str, EvalMetrics]
    aggregate: Dict[str, float]
    policy_id: str
    policy_source: str

class OptimisationEvaluator:
    def __init__(self, algorithm_runner, instances, metric_config, safety_guard)
        :
        self.algorithm_runner = algorithm_runner
        self.instances = instances
        self.metric_config = metric_config
        self.safety_guard = safety_guard

    def evaluate(self, policy, budget) -> EvalResult:
        results = {}
        for instance in self.instances:
            t0 = time.time()
            try:
                metrics = self.algorithm_runner.run(
                    instance=instance,

```

```

        policy=policy,
        budget=budget,
        safety_guard=self.safety_guard,
    )
except Exception as e:
    metrics = EvalMetrics(
        objective=math.inf,
        best_bound=None,
        gap=math.inf,
        primal_residual=math.inf,
        dual_residual=math.inf,
        violation=math.inf,
        integrality_gap=None,
        runtime=time.time() - t0,
        iterations=0,
        memory_mb=None,
        failed=True,
        failure_reason=repr(e),
    )
    results[instance.name] = metrics

aggregate = self._aggregate(results)
score = self._score(aggregate)
return EvalResult(
    score=score,
    metrics_by_instance=results,
    aggregate=aggregate,
    policy_id=getattr(policy, "policy_id", "unknown"),
    policy_source=getattr(policy, "source", ""),
)

def _aggregate(self, results: Dict[str, EvalMetrics]) -> Dict[str, float]:
    # Use robust statistics in implementation: median, IQR, fail rate.
    vals = list(results.values())
    fail_rate = sum(m.failed for m in vals) / max(len(vals), 1)
    return {
        "median_gap": median_safe([m.gap for m in vals]),
        "median_violation": median_safe([m.violation for m in vals]),
        "median_runtime": median_safe([m.runtime for m in vals]),
        "median_primal_residual": median_safe([m.primal_residual for m in

```

```

        vals]),
        "median_dual_residual": median_safe([m.dual_residual for m in vals]),
        "fail_rate": fail_rate,
    }

def _score(self, agg: Dict[str, float]) -> float:
    w = self.metric_config
    def log1p_safe(x):
        if x is None or not math.isfinite(x):
            return 1e6
        return math.log1p(max(x, 0.0))
    return -(
        w.gap * log1p_safe(agg["median_gap"])
        + w.violation * log1p_safe(agg["median_violation"])
        + w.runtime * log1p_safe(agg["median_runtime"])
        + w.primal_residual * log1p_safe(agg["median_primal_residual"])
        + w.dual_residual * log1p_safe(agg["median_dual_residual"])
        + w.fail * agg["fail_rate"]
    )

def median_safe(xs):
    xs = sorted([x for x in xs if x is not None and math.isfinite(x)])
    if not xs:
        return math.inf
    n = len(xs)
    return xs[n // 2] if n % 2 else 0.5 * (xs[n // 2 - 1] + xs[n // 2])

```

5.4 核心 metrics

不同问题类型应记录不同指标，但总表统一如下：

指标	定义	必须
Objective gap	与最优值、best known value 或强 solver 结果的相对差距	是
Primal residual	$\ Ax - b\ $ 、 $\ Ax + s - b\ $ 或 solver 定义的 primal infeasibility	是
Dual residual	KKT stationarity 或 operator-splitting dual infeasibility	是
Constraint violation	离散/连续约束违反程度	是

Integrality gap	MILP incumbent 与 relaxation/bound 差距	MILP 必须
Runtime	wall-clock time	是
Iteration count	达到 tolerance 的迭代数	是
Fail rate	NaN、发散、超时、无可行解、solver error 比例	是
OOD scaling	train-small-test-large 的性能保持	是
Policy complexity	代码行数、分支数、动作数、LLM cost	建议

6 Benchmark 获取与使用方案

6.1 Benchmark 分层

本项目不要一开始跑所有组合优化任务。建议分四层推进：

1. **Layer 0: Synthetic sanity check**。小规模 LP/QP/LASSO/graph Laplacian，验证 DSL、verifier、evaluator 正确。
2. **Layer 1: Relaxation benchmark**。MIPLIB/LLM-LNS 数据的 LP/QP 松弛，用 PDHG/ADMM/FISTA 加速 root relaxation 或 repair subproblem。
3. **Layer 2: Professional solver integration**。HiGHS、OSQP、SCIP/PySCIPOpt、OR-Tools 接入，对比专业 solver 的默认策略。
4. **Layer 3: MILP solving pipeline**。把 LLM-generated unrolled module 接入 LNS、relaxation repair 或 root-node preprocessing，评价对 final incumbent、bound、MIP gap 的影响。

6.2 推荐数据源

Benchmark	用途	获取/使用建议
Synthetic LP/QP	单元测试、可控条件数/稀疏度	自己生成；保存为 .npz 或 MPS/LP 格式
LASSO/Elastic Net	FISTA/PGD sanity check	sklearn/scipy 生成；有明确 objective gap
Graph relaxations	MIS/MaxCut/vertex cover 连续松弛	networkx 生成 ER/BA/RGG 图；输出 LP/QP relaxation
LLM-LNS data	MILP 与 LLM-LNS 对齐	使用其 IS、MIKS、MVC、SC 等数据；先做小规模子集

MIPLIB 2017	标准 MILP bench- mark	先跑 small/medium subset; root LP relaxation 优先
Ecole generators	可控 MILP 实例族	set cover、combinatorial auction、 capacitated facility location 等
HeuriGym	LLM heuristic bench- mark 参考	借鉴 agentic evaluator, 不建议第 一阶段全接入
OR-Tools examples	CP- SAT/PDLP/routing baseline	用作工程级 sanity benchmark

6.3 Benchmark 获取脚本建议

```
# scripts/setup_external.sh
mkdir -p external data results
cd external

git clone https://github.com/thuiar/LLM-LNS.git || true
git clone https://github.com/FeiLiu36/EoH.git || true
git clone https://github.com/ai4co/reevo.git || true
git clone https://github.com/NetSysOpt/PDHG-Net.git || true
git clone https://github.com/osqp/osqp-python.git || true
git clone https://github.com/ERGO-Code/HiGHS.git || true
git clone https://github.com/scipopt/PySCIPOpt.git || true
git clone https://github.com/ds4dm/ecole.git || true
git clone https://github.com/google/or-tools.git || true
git clone https://github.com/cornell-zhang/heurigym.git || true
```

MIPLIB 建议不要全量下载后立刻跑。第一阶段维护一个 `miplib_small.txt`, 只包含 20–50 个中小实例。随后再逐步扩到 benchmark subset。

6.4 训练/验证/测试划分

对每个 benchmark 应至少划分三类实例：

1. **Search-train**: LLM/evolution 在这些实例上得到 evaluator feedback。
2. **Validation**: 用于候选筛选和 early stopping, 不能直接作为 prompt feedback。
3. **OOD-test**: 规模更大、稀疏度不同、约束结构不同的实例, 只在最终报告中使用。

建议第一阶段采用如下划分：

任务	Search-train	Validation	OOD-test
Synthetic QP	$n = 100-500$	$n = 1000$	$n = 5000$
Set Cover LP	小规模 20 个	中规模 20 个	大规模 20 个
MIS relaxation	ER/BA 小图	同分布大图	不同图族大图
MIPLIB root LP	small subset	medium subset	benchmark selected subset

7 Baseline 使用方案

7.1 Baseline 层级

Baseline 必须分层，否则容易被质疑“LLM 只是昂贵 random search”。建议至少包括以下五层：

Baseline 类型	具体方法	必须程度
Vanilla algorithm	vanilla PGD/FISTA/PDHG/ADMM/PCG, 不做自适应	必须
Hand-crafted adaptive	residual balancing ADMM、adaptive restart FISTA、adaptive PDHG stepsize	必须
Professional solver	HiGHS、OSQP、SCIP/PySCIPOpt、OR-Tools PDLP/CP-SAT	必须
Black-box search	random search、grid、Bayesian optimisation、evolution without LLM	必须
Learned controller	MLP/LSTM schedule、unrolled learned scalar parameters、PDHG-Net 类方法	强烈建议
LLM heuristic	EoH、ReEvo、LLM-LNS、HeuriGym 风格 agent	根据任务选择
Ablation	LLM-only、LLM+verifier、LLM+evolution、LLM+verifier+evaluator	必须

7.2 各开源仓库在本项目中的角色

仓库	原始用途	本项目中的使用方式
LLM-LNS	LLM-driven LNS for MILP	强 baseline；借鉴 MILP 数据、LNS evaluator、SCIP 接口

EoH	LLM + evolutionary heuristic design	改造成 policy program evolution 框架
ReEvo	reflective evolution for hyper-heuristics	加入反思式候选改写与失败诊断
PDHG-Net	unrolled PDHG for LP	unrolling baseline; 借鉴 PDHG layer 与 two-stage inference
OSQP	ADMM-based QP solver	ADMM baseline; 比较 rho update policy
HiGHS	LP/QP/MIP open-source solver	专业 solver baseline; root LP/QP relaxation 对照
PySCIPOpt	SCIP Python interface	MILP pipeline、LNS、solver callback 实验
Ecole	ML-for-MILP control environment	生成 MILP 实例, 做 solver control 对照
OR-Tools	CP-SAT/PDLP/routing/graph algorithms	PDLP baseline, CP-SAT/routing 工程对照
HeuriGym	LLM heuristic benchmark	借鉴 agentic evaluator 和 query budget 设置

8 LLM Search Layer: 从 EoH/ReEvo 改造

8.1 候选生成对象

LLM 的生成对象不是完整 solver, 而是 **policy function**。对不同算法, policy 的作用不同:

- FISTA/PGD: 生成 stepsize、momentum restart、monotone correction。
- PDHG: 生成 τ, σ, θ schedule、primal-dual residual balancing、restart。
- ADMM: 生成 ρ update、over-relaxation、damping、termination warm rule。
- LNS-REPAIR: 生成 neighbourhood size、fixing ratio、relaxation repair trigger。
- PCG: 生成 restart、truncation、preconditioner selection 规则。

8.2 从 EoH/ReEvo 到 LLM4Unroll 的改动

模块	EoH/ReEvo 原始功能	LLM4Unroll 修改
Prompt	生成 heuristic 思路与代码	生成 DSL-constrained policy
Candidate program	heuristic function	update policy function
Evaluator	CO solution quality	optimisation metrics: gap/residual/violation/runtime/failure
Evolution mutation	语言层 heuristic 改写	policy rule mutation、threshold mutation、action mutation
Reflection	解释失败原因	基于 residual/gap/failure diagnostics 改写策略
Archive	保存优秀 heuristic	保存 policy、metrics、safe certificate、适用算法族

8.3 Prompt 模板

```

SYSTEM_PROMPT = """
You are designing a verifiable update policy for an unrolled operator-splitting
optimiser.

You must output a Python function policy(state: dict) -> dict following the DSL.
You are not allowed to solve the optimisation problem directly.
You are not allowed to import packages, use file I/O, network, exec, eval, or
loops.

Your policy must return a list of allowed actions and include a fallback-safe
behaviour.

Optimise for fast reduction of objective gap, primal/dual residuals, constraint
violation,
and low failure rate under a fixed iteration/time budget.
"""

USER_PROMPT_TEMPLATE = """
Algorithm: {algorithm_name}
Problem family: {problem_family}
Allowed state features: {allowed_features}
Allowed actions: {allowed_actions}
Safety bounds: {safety_bounds}
Current baseline diagnostics:
{baseline_diagnostics}

```

```

Previous best policies:
{archive_summary}

Please generate one new policy. Return only Python code for policy(state: dict)
    -> dict.
"""

```

8.4 Reflection 模板

```

REFLECTION_PROMPT = """
The previous candidate failed or underperformed.
Diagnostics:
- median gap: {median_gap}
- median violation: {median_violation}
- median primal residual: {median_primal_residual}
- median dual residual: {median_dual_residual}
- fail rate: {fail_rate}
- common failure reasons: {failure_reasons}

Analyse the likely cause in terms of stepsize, penalty update, restart, damping,
residual balancing, or over-aggressive actions. Then generate a safer and more
robust DSL-constrained policy. Return only code.
"""

```

9 Verifier 与 Safety Guard

9.1 静态 verifier

静态 verifier 对 LLM 代码执行前进行检查：

1. AST 检查：禁止 import、open、exec、eval、compile、globals、locals。
2. 复杂度限制：最大代码行数、最大分支数、禁止循环或限制循环上界。
3. 函数签名：必须有 policy(state: dict) -> dict。
4. 特征白名单：只能访问 algorithm registry 允许的 state fields。
5. 动作白名单：只能返回 ActionSpec 中的动作。
6. 数值边界：动作参数必须在上下界内，越界时自动 clamp 或 reject。

7. fallback 检查：必须在异常/不确定场景下调用 fallback 或保守动作。

9.2 运行时 safety guard

运行时 guard 在每次迭代后检查：

- 是否出现 NaN/Inf;
- residual 是否相对上一轮爆炸超过阈值;
- objective/gap 是否连续恶化;
- 参数是否越界;
- solver 是否报错;
- 单实例是否超时。

如果触发 guard，系统执行 fallback(), 恢复到 vanilla 或 hand-crafted safe rule。

10 算法 Runner 设计

10.1 统一 Runner 接口

所有算法实现应采用统一接口：

```
class UnrolledAlgorithmRunner:
    def __init__(self, algorithm_name, default_config):
        self.algorithm_name = algorithm_name
        self.default_config = default_config

    def initialise_state(self, instance):
        raise NotImplementedError

    def base_update(self, state, instance, params):
        """One vanilla operator-splitting update."""
        raise NotImplementedError

    def extract_features(self, state, instance):
        """Return whitelisted, normalised DSL state."""
        raise NotImplementedError

    def apply_actions(self, params, actions):
```

```

    """Update stepsize, penalty, restart flag, damping, etc."""
    raise NotImplementedError

def run(self, instance, policy, budget, safety_guard):
    state = self.initialise_state(instance)
    params = self.default_config.copy()
    history = []
    for k in range(budget.max_iters):
        dsl_state = self.extract_features(state, instance)
        policy_output = policy(dsl_state)
        actions = safety_guard.verify_runtime_actions(policy_output)
        params = self.apply_actions(params, actions)
        state = self.base_update(state, instance, params)
        metrics = self.compute_metrics(state, instance)
        history.append(metrics)
        if safety_guard.should_fallback_or_stop(history):
            params = self.default_config.safe_fallback()
    return self.final_metrics(history, state, instance)

```

10.2 PDHG Runner

PDHG 适合处理 saddle-point 形式:

$$\min_x f(x) + g(Kx), \quad (5)$$

其典型更新为

$$y^{k+1} = \text{prox}_{\sigma g^*}(y^k + \sigma K \bar{x}^k), \quad (6)$$

$$x^{k+1} = \text{prox}_{\tau f}(x^k - \tau K^\top y^{k+1}), \quad (7)$$

$$\bar{x}^{k+1} = x^{k+1} + \theta(x^{k+1} - x^k). \quad (8)$$

LLM 可以设计 $\tau_k, \sigma_k, \theta_k$ 和 restart/damping 规则。

10.3 ADMM Runner

ADMM 适合处理

$$\min_{x,z} f(x) + g(z) \quad \text{s.t.} \quad Ax + Bz = c. \quad (9)$$

典型更新为

$$x^{k+1} = \arg \min_x \mathcal{L}_\rho(x, z^k, u^k), \quad (10)$$

$$z^{k+1} = \arg \min_z \mathcal{L}_\rho(x^{k+1}, z, u^k), \quad (11)$$

$$u^{k+1} = u^k + Ax^{k+1} + Bz^{k+1} - c. \quad (12)$$

LLM 可以设计 ρ_k 、over-relaxation、residual balancing 和 damping。

10.4 FISTA Runner

FISTA 适合处理复合问题

$$\min_x f(x) + h(x), \quad f \text{ smooth}, \quad h \text{ proximal}. \quad (13)$$

LLM 可以设计 restart、momentum correction 和 stepsize schedule。

11 与专业求解器结合三条路线

11.1 路线 A: Root LP/QP Relaxation Accelerator

这是最推荐的第一条路线。许多 MILP 求解流程需要求解 root LP relaxation 或节点 LP/QP relaxation。你可以把 PDHG/ADMM 作为可展开一阶 solver，用 LLM 设计 schedule，然后与 HiGHS、OR-Tools PDLP、OSQP 对比。

优点：unroll 味道强，baseline 清楚，运行成本可控。

实验指标：root relaxation gap、residual、time-to-tolerance、warm-start quality、后续 MILP bound 改善。

11.2 路线 B: Relaxation-guided LNS Repair

把 LLM-LNS 的 neighbourhood selection 与你的 unrolled relaxation repair 结合：LNS 产生子问题后，用 PDHG/ADMM/FISTA 快速求 relaxation solution，再引导 rounding/repair。

优点：组合优化求解器味道强，容易与 LLM-LNS 做强对照。

实验指标：incumbent objective、MIP gap、time-to-best、repair success rate。

11.3 路线 C: Solver Internal Policy Module

通过 PySCIPOpt/Ecole 把 policy 接到 solver control 环节，例如 primal heuristic、node relaxation、cut/heuristic trigger。第一阶段不建议做太深，因为工程成本高，但可以作为博士课题第二阶段。

12 理论目标

12.1 Safe Policy Class

定义 safe policy class Π_{safe} : 所有 $\pi \in \Pi_{\text{safe}}$ 输出动作满足参数有界、更新有 damping、触发发散时 fallback。

定义 1 (安全策略类). 给定算法模板 $\mathcal{T}_\theta$, 若策略 π 在任意状态 s_k 下输出的参数 $\theta_k = \pi(s_k)$ 满足:

$$\theta_k \in \Theta_{\text{safe}}, \quad \|z_{k+1} - z_k\| \leq C_u, \quad \text{if } R_{k+1} > \gamma R_k \text{ then fallback}, \quad (14)$$

则称 π 属于 Π_{safe} 。

定理目标 1 (有界性). 对 *FISTA/PDHG/ADMM* 中的任一模板, 若 $\pi \in \Pi_{\text{safe}}$ 且 *problem data* 有界, 则由策略生成的展开序列 $\{z_k\}_{k=0}^K$ 在有限 *horizon* 内有界。

定理目标 2 (不劣于 fallback 的失败率控制). 若 *safety guard* 在 *residual explosion* 或 *NaN/Inf* 时切回 *fallback policy*, 且 *fallback policy* 在预算 B 内失败率为 δ_0 , 则 *LLM policy* 的 *catastrophic failure rate* 可由 δ_0 加上 *verifier* 漏检概率控制。

命题目标 1 (残差平衡策略的稳定窗口). 对 *ADMM/PDHG* 类 *residual balancing* 策略, 若步长/惩罚参数被限制在 $[\theta_{\min}, \theta_{\max}]$, 且每次乘法调整因子位于 $[a, b]$, 其中 $0 < a < 1 < b < \infty$, 则任意有限 *horizon* 内参数不会数值爆炸; 进一步可分析 *residual ratio* 被拉回给定窗口的次数上界。

12.2 不需要证明 LLM 本身

注意, 论文不需要证明 LLM 的“智能性”。理论部分只需要证明:

如果 LLM 生成的策略通过 *verifier* 并属于 *safe policy class*, 则算法保留某些稳定性、可行性或失败率控制性质。

这能避免陷入“如何证明 LLM 搜索有效”的困难。

13 实验计划

13.1 Phase 1: 最小可运行 Demo

- 算法: PDHG、ADMM。
- 问题: synthetic LP/QP、set cover LP relaxation、MIS LP/QP relaxation。
- LLM 生成对象: PDHG stepsize/restart、ADMM rho update。

- Baseline: vanilla、hand-crafted adaptive、random/evolution search、OSQP/HiGHS。
- 输出: gap/residual/time/fail rate 表格和 convergence curves。

13.2 Phase 2: 接入 LLM-LNS 与 MIPLIB

- 使用 LLM-LNS 的 IS/MIKS/MVC/SC 数据作为 MILP benchmark。
- 提取 root LP relaxation 或 LNS 子问题 relaxation。
- 比较 LLM-LNS heuristic、你的 LLM4Unroll relaxation-repair、二者组合。
- 对 MIPLIB small subset 跑 root relaxation 加速。

13.3 Phase 3: Professional Solver Pipeline

- PySCIPOpt: 把 relaxation-repair 模块接入 primal heuristic 或 LNS callback。
- HiGHS/OR-Tools: 比较 LP/QP relaxation 的求解速度和精度。
- Ecole: 构造可控实例族, 验证 train-small-test-large。

13.4 Phase 4: 论文级 Ablation

必须包含:

1. Without verifier;
2. Without reflection;
3. Without evolution;
4. LLM-only one-shot;
5. Evolution without LLM;
6. Random/BO with same query budget;
7. Safe fallback only;
8. Different LLM models / temperature / prompt variants;
9. Train-small-test-large;
10. Cross-problem transfer: set cover 训练, MVC/MIS 测试, 或 synthetic 训练, MIPLIB 测试。

14 可发表故事线

14.1 论文主张

建议论文主张如下：

Existing LLM-based optimisation methods mainly generate heuristics, while existing algorithm-unrolling methods rely on manually designed update structures. We bridge these two lines by introducing a verifiable DSL and an LLM-guided evolutionary search framework for designing unrolled operator-splitting optimisers used inside combinatorial optimisation solvers.

中文表述：

现有 LLM 优化算法设计主要生成组合优化启发式，现有 algorithm unrolling 方法主要依赖人工设计展开结构。本文提出一个可验证 DSL 与 LLM-guided evolutionary search 框架，自动设计组合优化专业求解器中 operator-splitting 模块的展开策略。

14.2 预期贡献

1. **DSL**: 提出一个面向 operator-splitting unrolling 的安全策略 DSL。
2. **Algorithm registry**: 系统整理可展开原算法及其可改组件。
3. **Search framework**: 将 EoH/ReEvo 风格 LLM evolution 改造成 optimisation-policy search。
4. **Optimisation evaluator**: 用 gap/residual/violation/runtime/fail rate 替代 heuristic evaluator。
5. **Solver integration**: 面向 MIPLIB、LLM-LNS、Ecole、HiGHS、OSQP、SCIP 的 benchmark 和 baseline 体系。
6. **Theory**: 证明 safe policy class 的有限 horizon 稳定性和 fallback-controlled failure。

15 开发里程碑

阶段	工作内容	产出
----	------	----

第 1-2 周	整 理 外 部 仓 库， 跑 环境脚本、baseline log 通 EoH/ReEvo/LLM- LNS/PDHG/OSQP/HiGHS 最 小例子	
第 3-4 周	实现 DSL schema、algorithm reg- istry、static verifier、runtime guard	可执行 DSL policy
第 5-6 周	实现 PDHG/ADMM/FISTA runner 与 optimisation evaluator	synthetic LP/QP demo
第 7-8 周	接入 EoH/ReEvo search loop，生成 第一批 policy	candidate archive、搜索曲线
第 9-10 周	接入 LLM-LNS 数据、MIPLIB small subset、HiGHS/OSQP baseline	benchmark result table
第 11-12 周	做 ablation、OOD scaling、failure analysis	paper demo version
第 13 周以 后	接入 PySCIPOpt/Ecole，做 profes- sional solver pipeline	第二阶段论文/博士课题扩展

16 给 Coding Agent 的超级提示词草案

下面是可直接改写后交给 coding agent 的超级提示词。

你是一个科研代码构建 agent。请基于我本地已经下载好的 external 仓库：LLM-LNS、EoH、ReEvo、PDHG-Net、OSQP、HiGHS、PySCIPOpt、Ecole、OR-Tools、HeuriGym，构建一个名为 llm4unroll 的 Python 项目。

项目目标：实现 Language-Guided Verifiable Algorithm Unrolling for Operator-Splitting Optimisers。LLM 不直接求解组合优化问题，而是离线生成可执行、可验证、可评估的 update policy，用于 PDHG、ADMM、FISTA/PGD 等 operator-splitting/unrolled optimisers。系统要面向组合优化专业求解器中的 LP/QP relaxation、LNS repair、root-node relaxation 和 MILP 子问题。

请按以下顺序构建：

1. 创建项目结构：

- src/llm4unroll/registry
- src/llm4unroll/dsl
- src/llm4unroll/algorithms
- src/llm4unroll/search
- src/llm4unroll/evaluator

- src/llm4unroll/solvers
- src/llm4unroll/benchmarks
- src/llm4unroll/experiments
- configs, scripts, data, results

2. 实现 DSL:

- schema.py: 定义 StateSpec、ActionSpec、SafetySpec、PolicySpec、PolicyOutput。
- primitives.py: 定义 allowed actions: set_tau, set_sigma, scale_tau, scale_sigma, set_rho, scale_rho, set_momentum, restart, damp, clip_update, switch_operator, fallback。
- verifier.py: 用 Python AST 检查 policy(state: dict) -> dict, 禁止 import、open、exec、eval、network、file I/O、global mutation; 检查只访问 allowed state features, 只返回 allowed actions; 检查参数范围。
- guards.py: 运行时检查 NaN/Inf、residual explosion、objective/gap 连续恶化、超时, 必要时 fallback。

3. 实现 algorithm_registry.py:

- 把 FISTA、PDHG、ADMM、DRS、ALM、PCG、LNS_REPAIR 全部注册进 ALGORITHM_REGISTRY。
- 每个算法包含 description、allowed state features、required features、allowed actions、safety bounds。
- 第一阶段重点支持 PDHG、ADMM、FISTA; PCG 和 LNS_REPAIR 先保留接口。

4. 实现算法 runner:

- algorithms/base.py 定义 UnrolledAlgorithmRunner 统一接口。
- algorithms/pdhg.py 实现一个可运行的 PDHG runner, 支持 tau/sigma/restart/damping policy。
- algorithms/admm.py 实现一个可运行的 ADMM/QP runner, 支持 rho update/damping policy。
- algorithms/fista.py 实现一个 LASSO/FISTA runner, 支持 stepsize/momentum/restart policy。

5. 实现 optimisation evaluator:

- evaluator/metrics.py 定义 objective_gap、primal_residual、dual_residual、constraint_violation、integrality_gap、runtime、fail_rate。
- evaluator/optimisation_evaluator.py 接收 policy、runner、instances、budget, 返回 EvalResult。
- score 使用 median log gap + median log violation + runtime + fail_rate + complexity penalty。

6. 改造 EoH/ReEvo 搜索框架:

- search/eoh_adapter.py: 复用 EoH 的 population/evolution 思路, 但候选从 heuristic function 改成 policy(state) function。
- search/reevo_adapter.py: 复用 ReEvo 的 reflection 思路, 根据 evaluator diagnostics 生成改进 policy。
- search/prompts.py: 实现 system prompt、candidate generation prompt、reflection prompt。
- search/run_search.py: 给定 algorithm name、problem family、budget, 生成 candidates, 调用 verifier/evaluator, 保存 candidates archive。

7. 实现 benchmark:

- benchmarks/synthetic_lp_qp.py: 生成可控 LP/QP/LASSO 问题。
- benchmarks/graph_relaxations.py: 生成 set cover、MIS、MVC 的 LP/QP relaxation。
- benchmarks/llm_lns_data.py: 读取 external/LLM-LNS 中的数据; 第一阶段只读小规模 IS/SC/MVC。
- benchmarks/miplib.py: 预留 MIPLIB small subset loader, 支持 MPS/LP 文件路径列表。

8. 实现 solver interfaces:

- solvers/highs_interface.py: 用 highs.py 或命令行调用 HiGHS 获取 LP/QP/MIP baseline。
- solvers/osqp_interface.py: 用 osqp Python 接口获取 QP/ADMM baseline。
- solvers/pyscipopt_interface.py: 用 PySCIPOpt 获取 MILP baseline、root relaxation 或 LNS 子问题接口。
- solvers/ortools_interface.py: 预留 OR-Tools PDL/CP-SAT baseline。

9. 实现 baseline runner:

- vanilla PDHG/ADMM/FISTA
- hand-crafted adaptive PDHG/ADMM/FISTA
- random search policy
- evolution without LLM
- LLM one-shot without verifier
- LLM + verifier
- LLM + verifier + reflection/evolution
- professional solver baseline: HiGHS, OSQP, SCIP, OR-Tools where applicable

10. 实现最小实验:

- configs/pdhg_synthetic_lp.yaml

- configs/admm_synthetic_qp.yaml
- configs/fista_lasso.yaml
- configs/pdhg_setcover_relaxation.yaml
- scripts/reproduce_phase1.sh
- 输出 results/tables/phase1.csv 和 results/figures/convergence_curves.pdf

11. 质量要求:

- 所有候选 policy 都要保存源码、verifier 结果、evaluator metrics、失败原因。
- 所有实验固定随机种子。
- 所有外部 solver 调用要有 timeout。
- 不要让 LLM 生成的代码直接访问文件系统或网络。
- 实验表格至少包含 gap、violation、primal residual、dual residual、runtime、iterations、fail rate。

请先实现 Phase 1 的最小可运行版本，然后给出如何运行的命令和一个 sanity-check result table。

17 第一篇论文的建议题目与摘要

17.1 题目

Language-Guided Verifiable Schedule Design for Unrolled Operator-Splitting Solvers in Combinatorial Optimisation

17.2 摘要草案

We study how large language models can assist the design of unrolled optimisation algorithms used inside professional combinatorial optimisation solvers. Unlike existing LLM-based heuristic generation methods, our framework does not ask the LLM to solve combinatorial problems directly. Instead, it defines a verifiable domain-specific language for operator-splitting optimisers, where the LLM proposes executable update policies such as stepsize schedules, restart rules, penalty updates and residual balancing strategies. Each candidate is statically verified, protected by runtime safety guards, and evaluated by an optimisation evaluator measuring objective gap, primal-dual residuals, constraint violation, runtime and failure rate. We instantiate the framework on PDHG, ADMM and FISTA modules for LP/QP relaxations and relaxation-guided repair subproblems arising from MILP benchmarks. Experiments on synthetic relaxations, LLM-LNS MILP families and MIPLIB subsets show that language-guided verifiable policies improve over hand-crafted

adaptive rules, random/evolutionary search and standard unrolled baselines under the same evaluation budget, while maintaining low failure rates and better out-of-distribution scalability.

18 参考文献与代码资源

参考文献

- [1] B. Romera-Paredes et al. *Mathematical discoveries from program search with large language models*. Nature, 2024. Code: <https://github.com/google-deepmind/funsearch>.
- [2] F. Liu et al. *Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model*. ICML, 2024. Code: <https://github.com/FeiLiu36/EoH>.
- [3] H. Ye et al. *ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution*. NeurIPS, 2024. Code: <https://github.com/ai4co/reevo>.
- [4] H. Ye et al. *Large Language Model-driven Large Neighborhood Search for Large-Scale MILP Problems*. ICML, 2025. Code: <https://github.com/thuiar/LLM-LNS>.
- [5] B. Li et al. *PDHG-Unrolled Learning-to-Optimize Method for Large-Scale Linear Programming*. ICML, 2024. Code: <https://github.com/NetSysOpt/PDHG-Net>.
- [6] X. Chen et al. *Learning to Optimize: A Tutorial for Continuous and Mixed-Integer Optimization*. arXiv, 2024. <https://arxiv.org/pdf/2405.15251>.
- [7] B. Stellato et al. *OSQP: An Operator Splitting Solver for Quadratic Programs*. Mathematical Programming Computation, 2020. Code: <https://github.com/osqp>.
- [8] HiGHS optimisation solver. Code: <https://github.com/ERGO-Code/HiGHS>.
- [9] PySCIPOpt: Python interface for the SCIP Optimization Suite. Code: <https://github.com/scipopt/PySCIPOpt>.
- [10] Ecole: Extensible Combinatorial Optimization Learning Environments. Code: <https://github.com/ds4dm/ecole>.
- [11] Google OR-Tools. Code: <https://github.com/google/or-tools>.

- [12] H. Chen et al. *HeuriGym: An Agentic Benchmark for LLM-Crafted Heuristics in Combinatorial Optimization*. ICLR Benchmark, 2026. Code: <https://github.com/cornell-zhang/heurigym>.
- [13] A. Gleixner et al. *MIPLIB 2017: Data-Driven Compilation of the 6th Mixed-Integer Programming Library*. Mathematical Programming Computation, 2021. Website: <https://miplib.zib.de/>.